MSTOkG

Release 0.1

Subimal Deb, V. S. Prasannaa

Mar 13, 2026

CONTENTS:

1 API 3

2 Building the dataset 13

Python Module Index 17

Index 19

Add your content using `reStructuredText` syntax. See the reStructuredText documentation for details.

CHAPTER

ONE

API

Add your content using `reStructuredText` syntax. See the reStructuredText documentation for details.

`MSTOOptpar.`**`AggressiveRefinement`**(*mol*, *fname*, *trials=10*, *tol=1e-16*, *fci=True*, *itermax=100*)

Performs a local, gradient-free iterative search (hill-climbing) to fine-tune basis set parameters. This exploitation phase provides the precision necessary for scientific accuracy.

Parameters

- **mol** – Best molecule object from the global search phase.
- **fname** – Path to save the refined candidate.
- **trials** – Number of independent refinement trials.
- **tol** – Energy tolerance for local convergence.
- **fci** – Whether to include FCI results in the saved output.
- **itermax** – Max iterations permitted per refinement trial.

Example

```
>>> mol_best = loadBSI("MOBS_Li_msto-03g.pickle")['mol_CISD'].mol
>>> AggressiveRefinement(mol_best, "MOBS_Li_refined.pickle", trials=5)
```

`MSTOOptpar.`**`BasisReshape`**(*mol*, *k*)

Reshapes the STO-nG basis of the molecule to cardinal number k.

Parameters

- **mol** (`pyscf.gto.mole.Mole`) – the *pyscf* molecule object
- **k** (`int`) – Cardinal number for which the basis is to be reshaped.

Returns

the *pyscf* molecule object with reshaped basis

Return type

`pyscf.gto.mole.Mole`

Example

```
>>> element = 'H' # choose your element
>>> mol = moldict('sto-6g')[element] # get the default molecule in your basis
>>> mol.basis
{'H': [[0,
   [35.52322122, 0.00916359628],
   [6.513143725, 0.04936149294],
```

(continues on next page)

(continued from previous page)

```
   [1.822142904, 0.1685383049],
   [0.625955266, 0.3705627997],
   [0.243076747, 0.4164915298],
   [0.100112428, 0.1303340841]]]}
>>> mol1 = BasisReshape(mol, 7)
>>> mol1.basis # The new coefficients are appended as [1.0, 1.0] for the deficit␣
↪cardinal numbers
{'H': [[0,
   array([3.55232212e+01, 9.16359628e-03]),
   array([6.51314372, 0.04936149]),
   array([1.8221429, 0.1685383]),
   array([0.62595527, 0.3705628 ]),
   array([0.24307675, 0.41649153]),
   array([0.10011243, 0.13033408]),
   array([1., 1.])]]}
>>> mol2 = BasisReshape(mol1, 5)
>>> mol2.basis # The excess coefficients are removed.
{'H': [[0,
   array([3.55232212e+01, 9.16359628e-03]),
   array([6.51314372, 0.04936149]),
   array([1.8221429, 0.1685383]),
   array([0.62595527, 0.3705628 ]),
   array([0.24307675, 0.41649153])]]}
```

This molecule object can now be used to optimize the basis set or compute the energies.

`MSTOOptpar.`**`CreateMoleculesStdBasisSets`**(*molecules='H Li Be B C N O F H2 LiH HF'*, *basissets=['6-31g', 'ccpvdz']*, *forcedump=False*, *dirname='./'*)

Batch creation of molecule objects from standard basis sets

Parameters

- **molecules** (`str`) – A string of space separated molecules as specified in `MSTOOpt.moldict()`
- **basissets** (`list`) – A list of basis set names for which the molecule objects are computed.

Example

```
>>> CreateMoleculesStdBasisSets(molecules='H Li Be B C N O F H2 LiH HF', basissets=[
↪'6-31g', 'ccpvdz'])
```

`MSTOOptpar.`**`CrossOvers`**(*pop*)

Creates a new population of molecules using discrete parameter crossovers.

Iterates through pairs of parent molecules and swaps basis set parameters based on a random binary template. This allows the offspring to inherit specific shells from different parents.

Parameters

pop (*list*) – A list of [energy, molecule] pairs representing the current population.

Returns

A list of new [energy, molecule] pairs generated via crossover.

Return type

list

`MSTOOptpar.DumpStandardBasis(element, basis, dirname='./', forcedump=False)`

Create the molecule object for a molecule in a given basis and dump in a file. A new file will be created only if it does not exist. The format of the filename will be *BS_<element>_<basis>.pickle* and stored in the specified directory.

Parameters

- **element** (`str`) – A molecule defined in `MSTOOpt.moldict()`.
- **basis** (`str`) – A basis name defined in *basis_set_exchange package* for which the energy is to be computed.
- **dirname** (`str`) – The directory where the molecule object is to be dumped.

Example

```
>>> DumpStandardBasis('H', 'sto-4g', dirname='./')
```

`MSTOOptpar.FetchBestMol(element, k, dirname='./')`

Fetches the best available basis for optimization of *element* for cardinal number *k*.

The order is as follows for a cardinal number *k*:

- MSTO-kG
- STO-kG
- MSTO-(k-1)G
- STO-(k-1)G
- ..etc.

It is best to ensure that at least the STO-2G basis is available.

To do:

- if k<2 fetch the sto-2g basis within this function.

`MSTOOptpar.GenerateInitialPopulation(mol, Nmax, facs)`

Creates a population of molecules by mutations from a given molecule.

`MSTOOptpar.LogicalRefinement(element, Nmax, trials=20, tol=1e-16, trials_mu=15, trials_co=20, prefix='MOBS', dirname='./', fci=True, itermax=25, agg_trial=True, ga_trial=True, kmax=12)`

Refine the MSTO basis sets generated. For 3 <= k <= 11 if the FCI energy for a given k is larger than that for k-1, an optimization is performed.

Parameters

- **mol** (`pyscf.gto.mole.Mole`) – the *pyscf* molecule object
- **Nmax** (`int`) – Population of candidates to consider in the optimization
- **trials** (`str`) – The number of generations to run
- **tol** (`float`) – Tolerance of energy for convergence
- **tol_mu** – Number of mutations to consider under each trial
- **trials_co** (`int`) – Number of crossovers to consider after each mutation
- **prefix** (`str` (optional. Defaults to *MOBS*)) – Prefix of the pickled database
- **dirname** (`str` (optional. Defaults to *./*)) – Directory to look for the dumped database files.

Returns
Nothing

Return type
None

Example

```
>>> element = 'H'
>>> Nmax = 10
>>> LogicalRefinement(element, Nmax, trials=20, tol=1e-16, trials_mu=15, trials_
↪co=20, prefix = 'MOBS', dirname=dirname, klst=range(3, 12))
```

`MSTOOptpar.Mutate(pop, facs)`

Creates a population of molecules by applying random fluctuations to the best candidate.

> **Note**
>
> This function is deprecated. Using *Mutate_v2()* is preferred as it utilizes parameter gradients between individuals for more efficient exploration.

This operator takes the current best candidate and generates new individuals by perturbing basis set parameters and geometry using the *dmol()* function. The population is updated if a mutant achieves a lower CISD energy than the existing best candidate.

Parameters

- **pop** (*list*) – Current population of [energy, molecule] pairs.
- **facs** (*list*) – A tuple/list containing (basis_perturbation_factor, geometry_perturbation_factor).

Returns
The updated population sorted by energy.

Return type
list

`MSTOOptpar.Mutate_v2(pop, facs)`

Creates a population of molecules using interpolative mutations between individuals.

> **Note**
>
> This is the preferred mutation operator for MSTO optimization.

Unlike standard random mutation, this version calculates a "parameter gradient" between two existing individuals in the population. It then shifts the parameters of the better individual toward the other by a randomized fraction, allowing for a more directed search of the parameter space.

Parameters

- **pop** (*list*) – Current population of [energy, molecule] pairs.
- **facs** (*list*) – A list containing the scaling factor for the mutation step size.

Returns
The updated population containing successful mutants.

Return type
list

`MSTOOptpar.OptimizeElement(mol, Nmax, fname, trials=10, tol=1e-10, trials_mu=100, trials_co=100)`

Optimizes basis set coefficients using a hybrid Memetic Algorithm. Evolves a population of basis sets toward a lower RCISD ground state energy.

Parameters

- **mol** (`pyscf.gto.mole.Mole`) – Initial PySCF molecule object used as a template.
- **Nmax** (*int*) – Population size of candidates for global exploration.
- **fname** (*str*) – Path to the pickle file where the best candidate is updated.
- **trials** (*int*) – Number of major optimization cycles.
- **tol** (*float*) – Energy tolerance for population convergence (Hartree).
- **trials_mu** (*int*) – Number of mutation attempts per major cycle.
- **trials_co** (*int*) – Number of crossover operations per mutation trial.

Returns
Tuple of (final population, generation history, energy history).

Return type
tuple

Example

```
>>> element = 'Li'
>>> mol = moldict('sto-3g')[element]
>>> fname = "MOBS_Li_msto-03g.pickle"
>>> pop, gen, en = OptimizeElement(mol, 5, fname, trials=2)
```

`MSTOOptpar.dmol(mol, fac_basis, fac_r)`

Creates a new molecule object with randomized basis parameters and geometry.

This function acts as a high-level mutation operator, perturbing both the basis set exponents/coefficients and the atomic coordinates.

Parameters

- **mol** (*pyscf.gto.mole.Mole*) – The template PySCF molecule object.
- **fac_basis** (*float*) – Scaling factor for basis set perturbations.
- **fac_r** (*float*) – Scaling factor for atomic coordinate perturbations.

Returns
A deep copy of the molecule with modified parameters.

Return type
pyscf.gto.mole.Mole

`MSTOOptpar.dumpBSI(BSI_instance, fname='BS_EO_pickled')`

Dumps a computed basis set results instance into a pickle file.

Useful for checkpointing the Memetic Algorithm or saving the best candidate for further aggressive refinement.

Parameters

- **BSI_instance** (*dict*) – The dictionary returned by getEnergies containing 'HF', 'CISD', and the molecule object.

- **fname** (*str*) – The output filename.

`MSTOOptpar.getEnergies(mol, nroots=4, fci=False, cisd=True)`

Retrieve the Hartree-Fock, FCI, and CISD energies for a molecule. The method ensures the molecule object is built locally to maintain consistency in parallel environments.

Parameters

- **mol** (`pyscf.gto.mole.Mole`) – The molecule whose energies are to be determined.
- **nroots** (*int, defaults to 4.*) – Number of roots for the FCI and CI solvers.
- **fci** (*bool, defaults to False.*) – Whether to compute Full Configuration Interaction energy.
- **cisd** (*bool, defaults to True.*) – Whether to compute Configuration Interaction Singles and Doubles energy.

Returns

A dictionary containing computed energies and converged PySCF objects.

Return type

dict

- **HF**: Total Hartree-Fock energy (float).
- **mol_HF**: The converged Restricted Hartree-Fock (RHF) mean-field object.
- **FCI**: List of FCI root energies (if fci=True).
- **mol_FCI**: The PySCF FCI solver object (if fci=True).
- **CISD**: Total CISD energy of the first root (float).
- **mol_CISD**: The converged PySCF CISD object.

Example

```
>>> mol = moldict('sto-3g')['H2']
>>> energies = getEnergies(mol, cisd=True)
>>> print(f"HF Energy: {energies['HF']}")
>>> print(f"CISD Energy: {energies['CISD']}")
```

`MSTOOptpar.getcoeffs(mol)`

Takes in a *pyscf.gto.mole.Mole* object and returns the element and shell with its basis set.

Parameters

mol (*pyscf.gto.mole.Mole*) – A *pyscf.gto.mole.Mole* molecule

Returns

A tuple containing (1) elements and their shell identifiers (2) numpy array containing the STO-kG basis set

Return type

tuple

Example

```
>>> # choose your element
>>> element = 'H'
>>> # get the default molecule in your basis
>>> mol = moldict('sto-6g')[element]
```

(continues on next page)

(continued from previous page)

```
>>> getcoeffs(mol)
([['H', 0]],
array([[[3.55232212e+01, 9.16359628e-03],
     [6.51314372e+00, 4.93614929e-02],
     [1.82214290e+00, 1.68538305e-01],
     [6.25955266e-01, 3.70562800e-01],
     [2.43076747e-01, 4.16491530e-01],
     [1.00112428e-01, 1.30334084e-01]]]))
```

`MSTOOptpar.loadBSI(fname='BS_EO_pickled')`

Returns an instance of the dumped basis set computations. Use this to improve the coefficients further.

`MSTOOptpar.moldict(basis)`

Returns a *pyscf.gto.mole.Mole* molecule dictionary for the chosen basis. The atom positions are from standard data. The elements/molecules are limited and can be improved. This is to provide a ready database of molecules.

Parameters

basis (*string*) – a string that specifies the standard basis set. It is expected to be a minimal basis set.

Returns

A dictionary of molecules with standard parameters.

Return type

dict

Example

```
>>> # choose your element
>>> element = 'H'
>>> # get the default molecule in your basis
>>> mol = moldict('sto-6g')[element]
>>> type(mol)
pyscf.gto.mole.Mole
>>> moldict('sto-6g')
{'H': <pyscf.gto.mole.Mole at 0x78fd225c96d0>,
'He': <pyscf.gto.mole.Mole at 0x78fd225c9810>,
'Li': <pyscf.gto.mole.Mole at 0x78fcae2851d0>,
'Be': <pyscf.gto.mole.Mole at 0x78fcae285310>,
'B': <pyscf.gto.mole.Mole at 0x78fcae285090>,
'C': <pyscf.gto.mole.Mole at 0x78fcae284f50>,
'N': <pyscf.gto.mole.Mole at 0x78fcae284e10>,
'O': <pyscf.gto.mole.Mole at 0x78fcae284cd0>,
'F': <pyscf.gto.mole.Mole at 0x78fcae284b90>,
'H2': <pyscf.gto.mole.Mole at 0x78fcae284a50>,
'HF': <pyscf.gto.mole.Mole at 0x78fcae284910>,
'LiH': <pyscf.gto.mole.Mole at 0x78fcae2847d0>,
'H2O': <pyscf.gto.mole.Mole at 0x78fcae284690>}
```

`MSTOOptpar.putcoeffs(iden, b)`

Creates a new basis set for a *pyscf.gto.mole.Mole* object from the element/shell identifiers and the array of basis coefficients. These need not be created by hand (you can if you want to and know what you are doing). This function was created to modify the basis of the molecule object and then inject the modified basis back into the molecule. The best way to use it would be to extract the identifier and basis set array using `MSTOkG.MSTOOpt.`

`getcoeffs()`, modify the basis appropriately and then use this function to generate the new basis. This new basis should be used to replace the basis in the *pyscf.gto.mole.Mole* object (and subsequently build it).

Parameters

- **iden** (*list*) – A list of element name and shell identifiers
- **b** (*numpy.array*) – Basis set coefficients.

Returns

Basis set for *pyscf.gto.mole.Mole* object

Return type

numpy.ndarray

Example

```
>>> element = 'LiH' # choose your element
>>> mol = moldict('sto-2g')[element] # retrieve the molecule with standard␣
↪parameters in the sto-2g basis
>>> iden, basis = getcoeffs(mol) # extract the identifiers and the basis set␣
↪coefficients
>>> iden
[['H', 0], ['Li', 0], ['Li', 0], ['Li', 1]]
>>> basis
array([[[1.30975638, 0.4301285 ],
    [0.23313597, 0.67891353]],
   [[6.16384503, 0.4301285 ],
    [1.09716131, 0.67891353]],
   [[0.24591632, 0.04947177],
    [0.06237087, 0.96378241]],
   [[0.24591632, 0.51154071],
    [0.06237087, 0.6128199 ]]])
>>> basis[0][1][0] # changing this element to 0.222222
np.float64(0.2331359749)
>>> new_basis = basis[:] # creating a copy
>>> new_basis[0][1][0] = 0.222222 # modifying the copy
>>> new_basis # let's check
array([[[1.30975638, 0.4301285 ],
    [0.222222  , 0.67891353]],
   [[6.16384503, 0.4301285 ],
    [1.09716131, 0.67891353]],
   [[0.24591632, 0.04947177],
    [0.06237087, 0.96378241]],
   [[0.24591632, 0.51154071],
    [0.06237087, 0.6128199 ]]])
>>> new_basis_set = putcoeffs(iden, basis) # create the basis from the new␣
↪coefficients
>>> from copy import deepcopy # we need this to create a copy of the molecule
>>> mol1 = deepcopy(mol) # `mol1` is the new copy
>>> mol1.basis = new_basis # modify the basis of the new molecule
>>> mol1.basis # now build this molecule and proceed with your computations
array([[[1.30975638, 0.4301285 ],
    [0.222222  , 0.67891353]],
   [[6.16384503, 0.4301285 ],
    [1.09716131, 0.67891353]],
```

(continues on next page)

(continued from previous page)

```
  [[0.24591632, 0.04947177],
   [0.06237087, 0.96378241]],
  [[0.24591632, 0.51154071],
   [0.06237087, 0.6128199 ]]])
```

`MSTOOptpar.randomize(a, fac)`

Add randomized fluctuations to a NumPy array by a defined factor.

This is used to introduce diversity in basis set parameters by perturbing exponents or coefficients.

Parameters

- **a** (*numpy.ndarray*) – The input array (e.g., basis set coefficients).
- **fac** (*float*) – The fluctuation scaling factor.

Returns

Perturbed array with values scaled by (1 + fac * random_offset).

Return type

numpy.ndarray

CHAPTER

TWO

BUILDING THE DATASET

This is the code that has been used to build the MSTO-KG basis sets.

Listing 1: Computing the MSTO-KG basis sets

```
import os
import numpy as np
import sys
import logging
import csv
from datetime import datetime
from copy import deepcopy
from joblib import Parallel, delayed

# Import your quantum chemistry modules
from MSTOkG.MSTOOptpar import *



def getonecandidate_local(candidate, fac=1e-1):
    try:
        mol = candidate['mol_CISD'].mol
        fac_basis, fac_r = np.random.random(2)*fac
        mol1 = dmol(mol, fac_basis, fac_r)
        en = getEnergies(mol1, nroots=4, fci=False, cisd=True)
        return en
    except Exception:
        return None

def AggressiveRefinement_v2(mol, fname, shots=10, tol=1e-8, itermax=10, n=3, passes=5,␣
↪mu=50, ncores=10):
    en_data = getEnergies(mol, nroots=4, fci=False, cisd=True)
    en0_val = en_data['CISD']
    facs = 1/10**np.arange(0, n+1)
    err = 1e9
    n_cores = ncores

    # Initialize CSV
    with open(csv_filename, mode='w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['Timestamp', 'Pass', 'Fac', 'Iteration', 'Tasks_Finished',
↪'CISD_Energy', 'Error'])
```

(continues on next page)

(continued from previous page)

```
    passno = 0
    while passno<passes:
        for fac_idx, fac in enumerate(facs):
            for shot in range(shots):

                Improved = True
                iters = 0
                while Improved and err > tol and iters < itermax:
                    faci = fac * np.random.random()
                    Improved = False
                    # total_tasks = shots * mu
                    total_tasks =  mu

                    tasks = (delayed(getonecandidate_local)(en_data, fac=faci) for _ in␣
↪range(total_tasks))
                    joblib_generator = Parallel(n_jobs=n_cores, return_as='generator
↪')(tasks)

                    tasks_finished = 0
                    for result in joblib_generator:
                        tasks_finished += 1

                        if result is not None:
                            if result['CISD'] < en_data['CISD']:
                                en_data = result
                                Improved = True
                                dumpBSI(deepcopy(getEnergies(en_data['mol_CISD'].mol,␣
↪fci=True)), fname=fname)
                                err = np.abs(en0_val - en_data['CISD'])
                                en0_val = en_data['CISD']

                                # Log improvement to the .log file
                                logging.info(f"IMPROVEMENT: Pass {passno}, Iter {iters}␣
↪| New Energy: {en_data['CISD']:.10f}")

                                # Log to CSV for plotting later
                                with open(csv_filename, mode='a', newline='') as f:
                                    writer = csv.writer(f)
                                    writer.writerow([datetime.now(), passno, faci, iters,
↪ tasks_finished, en_data['CISD'], err])
                                # iters=max([0,iters-1])
                                # Terminal Progress Update (Status Bar)
                                print(f"{element} | k = {k} | Pass {passno} shot: {shot}␣
↪| Progress: {tasks_finished}/{total_tasks} | Best E: {en_data['CISD']:.10f} | fac:
↪{faci} | err: {err:.4g}", end='\r', flush=True)

                        # Terminal Progress Update (Status Bar)
                        print(f"{element} | k = {k} | Pass {passno} shot: {shot} |␣
↪Progress: {tasks_finished}/{total_tasks} | Best E: {en_data['CISD']:.10f} | fac: {faci}
↪ | err: {err:.4g}", end='\r', flush=True)
```

(continues on next page)

(continued from previous page)

```
                    print()
                    logging.info(f"Finished Iteration {iters} of Pass {passno}")
                    iters += 1
        print(f"{element} | k = {k} | Pass {passno} shot: {shot} | Progress: {tasks_
↪finished}/{total_tasks} | Best E: {en_data['CISD']:.10f} | fac: {faci} | err: {err:.4g}
↪", end='\r', flush=True)
        passno =passno+1
    print()

if __name__ == "__main__":

    element = sys.argv[1]

    # --- Setup Logging ---
    log_filename = f"refinement_{element}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.log"
    csv_filename = f"convergence_{element}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv
↪"

    logging.basicConfig(
        level=logging.INFO,
        format='%(asctime)s [%(levelname)s] %(message)s',
        handlers=[logging.FileHandler(log_filename), logging.StreamHandler(sys.stdout)]
    )
    dirname = "./data/"
    if len(sys.argv)>2:
        klst = [int(k) for k in sys.argv[2:]]
    else:
        klst = range(2, 12)
    for k in klst:
        fname = os.path.join(dirname, f"MOBS_{element}_msto-{k:02d}g.pickle")
        mol = FetchBestMol(element, k, dirname=dirname)

        logging.info(f"Starting refinement for {element} (k={k})")
        AggressiveRefinement_v2(mol, fname, shots=2, tol=1e-8, itermax=20, n=10,␣
↪passes=10, mu=1000, ncores=15)
    logging.info("Refinement process complete.")
```

PYTHON MODULE INDEX

m

MSTOOptpar, 3

INDEX

A

AggressiveRefinement() (*in module MSTOOptpar*), 3

B

BasisReshape() (*in module MSTOOptpar*), 3

C

CreateMoleculesStdBasisSets() (*in module MSTOOptpar*), 4
CrossOvers() (*in module MSTOOptpar*), 4

D

dmol() (*in module MSTOOptpar*), 7
dumpBSI() (*in module MSTOOptpar*), 7
DumpStandardBasis() (*in module MSTOOptpar*), 4

F

FetchBestMol() (*in module MSTOOptpar*), 5

G

GenerateInitialPopulation() (*in module MSTOOptpar*), 5
getcoeffs() (*in module MSTOOptpar*), 8
getEnergies() (*in module MSTOOptpar*), 8

L

loadBSI() (*in module MSTOOptpar*), 9
LogicalRefinement() (*in module MSTOOptpar*), 5

M

module
    MSTOOptpar, 3
moldict() (*in module MSTOOptpar*), 9
MSTOOptpar
    module, 3
Mutate() (*in module MSTOOptpar*), 6
Mutate_v2() (*in module MSTOOptpar*), 6

O

OptimizeElement() (*in module MSTOOptpar*), 7

P

putcoeffs() (*in module MSTOOptpar*), 9

R

randomize() (*in module MSTOOptpar*), 11